UNIVERSITÀ DI PISA

Decision Support Systems - Laboratory of Data Science

Fabrizio Anelli, Emanuele Nardi, Marco Tamberi

December 2025

Chapter 1

Using Python

1.1 Assignment 1

The first step is always to perform data understanding: before making decisions, it is necessary to understand the data context. For this reason, we loaded both datasets in Python and checked for duplicates and missing values.

Next, we decided to create a simple visualization to improve data exploration by counting how many artists belong to each region.

Once we had a clear understanding of the data, we proposed additional potentially useful variables to implement, such as the *Aggressiveness* measure (a combination of loudness and BPM) and *debut_age*, which helps quickly visualize the age at which each artist debuted.

Finally, we noticed that the *disc_number* variable is highly imbalanced toward the value `1`.

1.2 Assignment 2

After completing the lighter exploratory phase, we moved on to hands-on data cleaning. First, we manually retrieved the missing birth provinces/places for a subset of artists through simple online searches, in order to enrich the available metadata.

Next, we implemented a geocoding function based on Nominatim (OpenStreetMap) which, given a place name, returns latitude, longitude, region/state, country, and an estimated nationality. We then applied this function to the birthplace of each artist in order to fill as many geographic fields as possible. Since the automatic geocoding for *Baby K* required a manual adjustment (born in Singapore), we fixed her geographic information by hand.

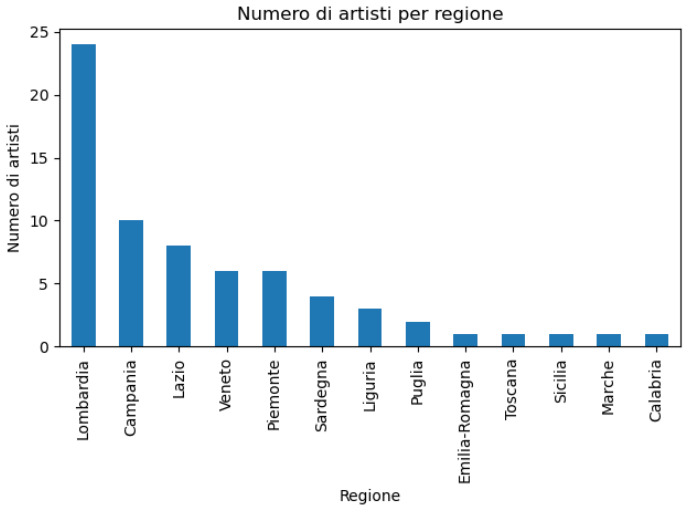

Figure 1.1: Distribution of Artist by Region

All the retrieved geographic information was stored in the `artists_geo` dictionary. By iterating over each artist in `artists.xml`, we cleaned and normalized the artist name, matched it with the corresponding key in `artists_geo`, and updated the XML fields while keeping track of how many artists and fields were updated. In addition, we computed and added an H3 index for each artist based on latitude and longitude.

As suggested in the *Decision Support Databases* course, we also converted date fields to the `YYYYMMDD` format for practicality, improving compatibility with joins and supporting time hierarchies in the Data Warehouse. Finally, we generated a new file `artists_cleaned.xml` containing all these updates, resulting in 36 artists enriched via geocoding and 335 added/filled fields.

Once `artists.xml` was cleaned, we proceeded with `tracks.json`. After loading the data, we focused on missing values: numeric fields were filled using the median, while categorical fields were filled using the mode. We also computed default median values for year, month, and day to be used as fallbacks when needed.

Our priority was to recover missing `year` values from `album_release_date` whenever available; if parsing failed or values were still missing, we fell back to the median defaults for `year`, `month`, and `day`. We then computed the median for each quantitative feature and the mode for each categorical feature, and iterated over every single track to replace missing values accordingly.

When `year`, `month`, or `day` were missing, we attempted again to extract

them from `album_release_date`, tracking how many date values were filled through the `date_riempite` counter; in case of errors, the script simply skipped the problematic record and continued. If the `album` field was missing, we copied it from `album_name`, also counting the number of performed updates.

We then fixed the `explicit` flag, which must be `True` whenever `swear_IT` or `swear_EN` is greater than zero. Finally, as required by the assignment, we created a new `season` variable based on the song release date. As a final step, we derived additional time keys in `YYYY`, `YYYYMM`, and `YYYYMMDD` formats to support time-based aggregations and hierarchical joins in the Data Warehouse.

1.3 Assignment 3

We chose to build the song categories based on the melodic information in contained in the data, more specifically using the attributes *bpm*, *rolloff*, *flux*, *rms*, *flatness*, *spectral_complexity*, *pitch* and *loudness*, that are all the ones we had at our disposal to describe the melodic and rhythmic components of the songs.

The categories were built using clustering. We tried several clustering methods: hierarchical clustering, HDBScan and K-Means.

For hierarchical clustering, we tried to compare complete, single and average linkage. However, we soon found out that both single and average linkage created a giant cluster with very few outliers, thus giving us one category that contained almost all the songs, and several outlier categories sparsely populated. Complete linkage gave us better results, but still the clusters were very imbalanced (based on the distance threshold used, some could have thousands of elements in it, some hundreds, or even less). We considered these results to be unsatisfactory for our purpose, so we discarded this option.

HDBScan performed in almost the same way as the single linkage. Based on the parameters we use for this clustering, either almost all the points are considered outliers, or almost all of them are put in the same big cluster, thus making this method of categorization useless in our setting.

We finally tried K-Means. First, we tried using a number of clusters (K) variable between 2 and 30, in order to see the Sum of Squared Error (SSE) and silhouette score of the clustering for each K. Using this information, we chose a K that was in the elbow of the SSE plot, gave us a high silhouette score (when compared to the other points in the proximity of the elbow of the SSE plot) and didn't contain too few clusters nor too many clusters. In the end, we opted for $K = 7$.

So we performed K-Means with 7 clusters and that gave clusters that

were much more balanced than the previous methods. We then looked into the average values of each cluster with respect to the attributes we chose for the clustering, in order to assign some kind of name to these categories:

1. Minimal: the cluster where songs had the lowest *spectral_complexity* (that also had the lowest *loudness*, *rms*, *flux* and one of the lowest *rolloff*);
2. Fast-Flow: where songs had the highest *bpm* (and also a high *spectral_complexity*);
3. Melodic: where songs had a high average *pitch*;
4. Slow Dark: where songs had the lowest *bpm* and *rolloff*;
5. Clean: where songs had the lowest *flatness*;
6. Warm Bangers: where songs had the highest *loudness* and *rms* (thus "Bangers") and the highest *pitch* (thus "Warm");
7. : Hype: where songs had the highest *spectral_complexity* and *rolloff* and high *loudness*

1.4 Assignment 4

For the data warehouse, we decided to create dimension tables for Album, Artist, Date, Symphony (the melodic and rhythmic information used also in Assignment 3) and Text, and a dimension table for the Artist Geographic information, connected directly to the Artist dimension. The idea was to also have hierarchies in the Date and Artist Geography dimensions:

1. for Date, it was Day → Month → Year. We opted not to include the Season in the hierarchy because using only the name of the season made it easier to perform future assignment that required the Season attribute, compared to using something like `YYYY-SEASON`;
2. for Artist Geography, it was City → Province → Region → Country. We did not include the h3 in this hierarchy because the h3 cells are not necessarily contained in one province or another (or region or country), for example an h3 cell containing a border between two provinces.

To handle the artist featured in a song, which is a many-to-many relationship, we created a bridge table. First we created a Feats dimension, that only contained primary keys (to be referenced from the fact table and the bridge table), then the bridge table Feats_Bridge that contains the foreign keys to the

Feats dimension (thus connecting the bridge table with the fact table) and the foreign keys to the Artist dimension, so that for each song there is a line for each featured artist (so, a song with 3 featured artists has 3 lines, a song with 0 featured artists has none).

Thus, the diagram for the final logical schema is in 1.2:

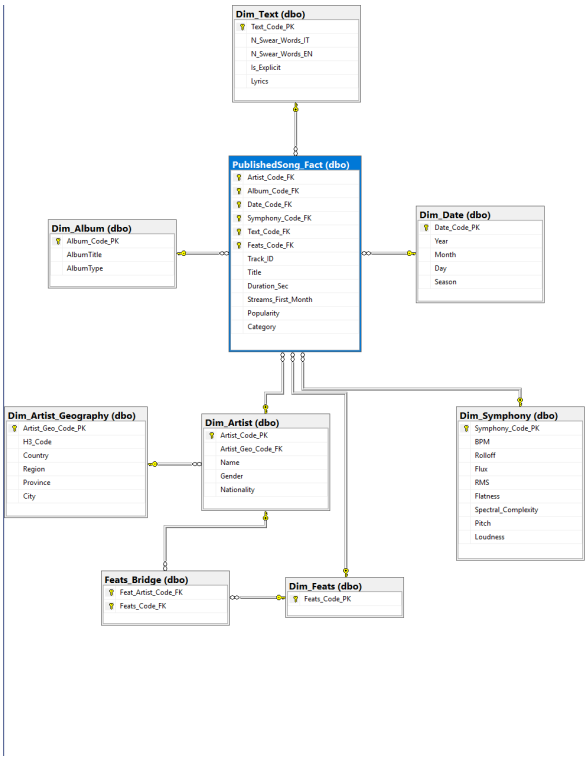

Figure 1.2: Diagram of the logical schema.

1.5 Assignment 5

The task of Assignment 5 required splitting the data into separate files in a way that is consistent with the tables defined in the Data Warehouse.

First, we loaded `tracks_cleaned.json` and `artists_cleaned.xml`. Then, we initialized a surrogate-key counter for each table and created in-memory buffers (lists) to store the rows that would later be exported to CSV files.

To avoid duplicates in the dimension tables and to ensure efficient linking from the fact table, we built lookup structures mapping *natural keys* to integer

surrogate keys. This ensures that when the same dimensional record appears again, we reuse the existing surrogate key rather than creating duplicate rows.

We also defined a `write_csv` function to generate a CSV file given a filename, a header, and a list of rows.

We then processed `artists_cleaned.xml`: for each `row` node, we retrieved the artist natural identifier (`id_author`) and the attributes required for `ArtistDim`. In parallel, we built `ArtistGeoDim` using the natural key tuple

$$\texttt{geo\_tuple} = (\texttt{country}, \texttt{region}, \texttt{province}, \texttt{city}, \texttt{h3})$$

from which geographic surrogate keys are generated (or retrieved). For each new artist we created a new primary key in `ArtistDim`; otherwise we reused the existing mapping. Additionally, we created a `name` $\rightarrow$ `pk_artist` mapping to later resolve featured artists, since they are provided as free-text strings in the tracks dataset.

Next, we loaded `tracks_cleaned.json` and iterated through each track. For `AlbumDim`, the natural key is `id_album`: for each new album we generate a new surrogate key, otherwise we reuse the existing one, and store the corresponding attributes (title, type, and release date).

We applied the same approach to `TextDim`, where the natural key is the track identifier (`id`). In this step we also cleaned the `lyrics` field by removing newline characters, producing a compact text that is more suitable for CSV export.

For `DateDim`, we used the triple `(year, month, day)` as the natural key: for each unique combination we generate a new surrogate key and derive time representations such as `YYYY`, `YYYYMM`, and `YYYYMMDD` (together with the `season` variable), which support joins and time hierarchies in the DW.

We followed the same principle for `SymphonyDim`, treating it as a 1:1 dimension with respect to each track and storing audio-related features (bpm, loudness, rolloff, etc.).

Regarding `GroupDim`, we chose to create a new group for every single track. This dimension contains only its surrogate key and acts as a container to link featured artists.

The bridge table `GroupFeatures` links `GroupDim` and `ArtistDim`: starting from the `featured_artists` text field, we split it into a list of artist names, and for each name we searched for a match in the `artist_name_lookup` mapping built from the XML. When a match is found, we add a row to the bridge table containing: the group FK and the featured artist FK

Finally, for the fact table we retrieved the main artist natural identifier (`id_artist`) and mapped it (when possible) to the corresponding surrogate key in `ArtistDim`. We then generated a surrogate primary key for the fact table

and populated the record with measures/attributes (e.g., duration, first-month streams, popularity, category) and all foreign keys pointing to the dimension tables.

Once all buffers were populated, we produced the final CSV files by calling `write_csv` for each Data Warehouse table.

1.6 Assignment 6

Now that we have generated the CSV files for each table, it is time to load the data into SSMS (SQL Server). First, we connect to the database through an ODBC driver: the `cnxn` object opens the database connection, while `cursor` creates a cursor used to execute SQL commands.

We then define a `read_csv` function that reads each CSV row as a dictionary, using the file header as the set of keys (i.e., column names).

Next, we implement one loading function for each dimension table. Before executing the `INSERT` statements, we enable `SET IDENTITY_INSERT <table> ON`, which allows us to manually insert values into an `IDENTITY` column (the auto-increment primary key). In SQL Server, `IDENTITY_INSERT` can be enabled for only one table at a time within the same connection.

After iterating over all CSV rows (casting the primary key to integer when required), we disable `IDENTITY_INSERT` using `SET IDENTITY_INSERT <table> OFF` and finalize the step with `cnxn.commit()`, which permanently saves the inserted records.

Following the same pattern, we load all dimension tables while keeping consistency between the CSV attributes and the Data Warehouse schema in SSMS. Once all loading functions are defined, we execute the load in a logical order to avoid foreign key conflicts: first the dimensions, then the bridge table, and finally the fact table.

Finally, we close both the cursor and the database connection.

1.7 Assignment 7

After creating the new _SSIS tables through a simple query, we moved on to SSIS. Our approach was to load the dimension tables first, and then apply a random 30% sampling from the fact table `PublishedSong_Fact` into `PublishedSong_Fact_SSIS` using the dedicated sampling component.

If we had sampled 30% independently from every table, there would have been a high risk of breaking referential consistency between primary and foreign keys: records would no longer match and joins would become partial or unusable.

Therefore, even if it may seem trivial, we decided to sample only the fact table, as required by the assignment.

Chapter 2

Using Microsoft SQL Server Integration Services

2.1 Assignment 8

We joined Fact table, Date and Artist dimensions, grouped by *Year* and *Name* and sorted the output.

In this flow and in the following ones, we opted to use the Lookup operator to perform inner joins when possible instead of the Merge Join, since it doesn't require the tables to be sorted.

2.2 Assignment 9

After joining the Fact table with the Artist, Artist Geography and Date dimension, we split it based on the *Season* to keep on one side only the songs released in the summer and on the other side those released in the winter. After the groupings, we re-join the two tables using an outer join in order not to lose the *Region*s which have songs released in summer or in winter, but not in both. Then we create the measures needed for the required output.

SSIS tells us that the final sorting is not needed because the table is already sorted by Region, but we kept it anyway as a selection of the attributes we want to show in the output.

2.3 Assignment 10

After joining the Fact table with the Artist and Artist Geography, we group by *Category* and *Region*. In this way, we can then use the Multicast to keep a copy of this table while also computing another Group By, this time only grouping on *Category*. Then, after joining the two tables on *Category*, we can create the ratio needed between the cumulative number of streams in a region for a certain category and the total number of streams for that category.

2.4 Assignment 11

After connecting Fact table and Date dimension, we searched for the date of the first song(s) released for each artist by grouping by artist and searching for MIN(*Day*). To do so we used a Multicast, in order to also keep a copy of the non-grouped table. In cases when an artist released more than one song on the date of their first release, we chose to use all of them when calculating the measures, since there was no way to choose one over the others.

We then joined the two tables and used a Conditional Split to keep, for each artist, only the first song(s) released.

In order to be able to compute the average number of streams of all the other artists, we decided to have a cartesian product of the table with itself, then for each artist we kept all the artists excluding themselves, and finally we did the grouping to obtain, for each artist, the average streams of all the other artists. The cartesian product was done by giving to all the elements of the table a fictitious key, that had value equal to 1 for everyone, then used a Multicast to have 2 copies of the same table and finally Merge Joined on the fictitious key.

To calculate the Standard Deviation, we added a Derived Column with the value of the number of streams squared, then we grouped by artist and calculated the average and count of the streams, in order to be able to use the following formula:

$$\sigma = \sqrt{\frac{1}{N}\sum (x^2 - \bar{x}^2)} \ . \tag{2.1}$$

Then we created two derived columns IsTrending and IsFlopping that assume value 1 when, respectively, the song is trending and flopping, 0 otherwise.

Apart from the edge case described in the assignment, we looked at one more edge case: when the artist has only 2 songs. The problem in this case is that if we define "trending" as having strictly more streams than the average plus the standard deviation, and "flopping" as having strictly less streams than the average minus the standard deviation, then none of the 2 songs will be

trending or flopping. But if we instead use more or equal streams for "trending" and less or equal for "flopping", then the song that has more streams will always be trending and the other one always be flopping. In order not t have a predetermined outcome for the 2 songs, we decided to use the same approach as the case in which the artist only has one song, thus comparing it with the streams for the first song(s) of all other artists.

Using a Multicast we were able to group by main artist and featured artist separately, then joined these two tables back together with an outer join (otherwise the output wouldn't include artists who didn't feature in any song even if they were the main artist in some songs).

2.5 Assignment 12

We chose to take a look at how much songs released as singles out-perform song released in another way (in an album or compilation, or if the method of release is unknown). To do so, for each artist, we averaged the values of *Streams_First_Month* for songs with *Album_type* = 'singles' and we averaged the values of *Streams_First_Month* for every other category, and then took the ratio of the two averages.

To implement this, after joining Fact table and Album dimension, we separated the data into the 4 types of the values of *Album_Type* ('singles', 'album', 'compilation', 'null') using the Conditional Split, calculated the average for the 'singles' and memorized sum and count of *Streams_First_Month* for the others, in order to be able to combine them and have the global average

$$(AVG_{non\_singles} = \frac{SUM_{album} + SUM_{compilation} + SUM_{null}}{COUNTalbum + COUNTcompilation + COUNTnull}). \tag{2.2}$$

Then we joined the tables back together using outer joins, so that we would not lose information, and set to 0 the null values resulting from the outer join. Finally we calculated $AVG_{non\_singles}$ as shown before, created the column with the ratio between 'singles' and 'non singles' as *Singles_Performance*, joined with the Artist dimension to have the name of the artists in the final output and sorted by *Singles_Performance*.

This information can be useful from a business point of view because it can help decide how to program the marketing for a certain artist: if an artist has a high *Singles_Performance*, it means that the listeners are more interested in the singles than the other songs, thus we know, for example, not to spend too much time or money marketing the songs that aren't singles, since it would likely be a waste.

We divided the *Album_Type* in 'singles', 'album', 'compilation' and 'null' instead of just 'singles' and 'non-singles' to be able to show the average stream of songs of each type in the output, together with the ratio computed before.

2.6 Assignment 13

Using the same "trending" concept used in previous assignment, but done with *popularity* instead of *Streams_First_Month*, we chose to take a look at the songs that were trending for each artist and how many featured artists it had compared to the average song of that artist. The idea is to look if the best performing songs of a certain artist include a higher or lower number of featured artist than usual, so that if the record label needs to push an upcoming album (usually for an artist that needs a boost due to recent flops or inactivity) we are able to suggest whether it is better to use a lot of featured artists or it is better to keep the number of features low.

To do so we left joined the Fact and the Bridge tables, in order to be able to count how many features each song has (the left join is needed in order not to lose the songs without any featured artists, for which instead we get a null values that we immediately set to 0). Then we calculate the average and standard deviation of the popularity of each artist's songs from the fact table (and joined with the Artist dimension to have the names of the artists), and the average number of features in their songs with the Fact and Bridge tables. To do the latter, we didn't use Lookup to join Fact and Bridge table, because Lookup would have filtered out the songs that didn't have a match, so we use a merge join. After joining these results, we separated the trending songs with a Conditional Split and created for them the measure *Delta_Feats*, that indicates the difference between the number of featured artists in the songs and the average for that artist.

Chapter 3

Using MDX, SQL Management Studio, and PowerBI

3.1 Assignment 14

In this assignment, we focused on the construction of the data cube. Besides the standard steps required to create the Data Source View, the cube, and the dimensions, the most critical aspect was the management of the many-to-many relationship between songs and artists, needed to correctly handle featured artists.

To achieve this, we modeled the many-to-many relationship through the bridge table and also created a dedicated Artist dimension without featured artists, which allows us to analyze both the set of all featured artists involved in a song and, at the same time, to clearly identify the main artist associated with each track.

Within the *Dimension Usage* tab, several manual adjustments were necessary, including the implementation of reference relationships and the correction of the relationships involving *feat bridge*.

Finally, a set of calculated measures that would help us in subsequent assignments was defined, namely *AvgWeightedStreams*, *StreamChange*, *StreamsByMinute*, *WeightedStreams*, and *PreviousStreams*.

Of those measures, only *PreviousStreams* was correctly implemented, while the other measures presented errors which made them not usable. This was only realized once we started working on the following assignments (which is also after

the cube was delivered, which is the reason these not working measures are still present in the cube).

The not working measures were then implemented directly in the MDX queries, and will be discussed in further detail in their description.

3.2 Assignment 15

Our MDX query reports the monthly value of `[Measures].[Streams First Month]` for each Italian region and includes an overall country total. We computed the overall total only for Italy because it's the only country in our dataset for which regional information is available.

Firstly, we define a calculated member `[Totale Italia]` on the `Region` hierarchy as the sum of the measure across all region members, explicitly excluding the `All` member to avoid double counting. In the `SELECT` statement, the measure is placed on columns, while rows are built as the Cartesian product between the set of months (again excluding the `All` member) and the set of regions without `All`, extended with the calculated `Totale Italia`. Finally, the `WHERE` clause restricts the query context to artists whose country is Italy.

3.3 Assignment 16

First we defined a calculated member to compute `[Measures].[Weighted Stream]` as the sum of `[Measures].[Streams First Month]` when the artist is the main artist, times 0,8, and `[Measures].[Streams First Month]` when the artist is a featured artist, times 0,2. Then another calculated member that divided the previous one by the sum of `[Measures].[Streams First Month]` in a certain year, calling it `[Measures].[Average Weighted Streams]`.

The `SELECT` has `[Measures].[Average Weighted Streams]` on columns and a non-empty cartesian product of the set of artists' names and the year, in order to avoid having null values in the output.

3.4 Assignment 17

The Assignment asked to compute, for each song category, the percentage increase or decrease in streams with respect to the previous year. We use the cube measure `[Measures].[PreviousStreams]` (which represents the streams value for the prior year in the same context) and define a calculated measure

[Measures].[Streams YoY %] as

$$\frac{\texttt{Streams First Month} - \texttt{PreviousStreams}}{\texttt{PreviousStreams}}.$$

Whenever PreviousStreams is empty or equal to zero, the expression returns NULL to avoid division-by-zero errors. We visualize the current-year streams, previous-year streams, and the YoY percentage side by side on columns. On rows, we have the Cartesian product between Category and Year (excluding the All members) and apply NON EMPTY to keep only combinations that actually have data.

3.5 Assignment 18

For this query, we use the calculated measure [Measures].[Previous Streams], that tracks the streams of the year before the current one. Thanks to this, we were able to define a calculated member [Measures].[IsGrowing] that returns NULL if

$$\texttt{[Measures].[Avg Weighted Streams]} - \texttt{[Measures].[Previous Streams]} \tag{3.1}$$

is less than 0, else it returns the percentage increase in streams calculated as

$$\frac{\texttt{[Measures].[Avg Weighted Streams]} - \texttt{[Measures].[PreviousStreams]}}{\texttt{[Measures].[Previous Streams]}}. \tag{3.2}$$

We decided to add also that [Measures].[Previous Streams] must be not empty, otherwise trying to calculate [Measures].[IsGrowing] with NULL as the value for the previous year would only yield more NULL values.

In the SELECT statement has on rows we put the cartesian product of year, season and category, excluding rows with null values for one of them or for [Measures].[IsGrowing].

3.6 Assignment 19

This query is designed to uncover non-trivial *seasonality effects* at the artist level. For each artist, we first compute [Measures].[Artist Season Avg], the average value of [Measures].[Streams First Month] across all seasons (excluding the All member). We then derive [Measures].[Delta vs Artist Avg %], the percentage deviation of a given season from the artist's own seasonal average. Finally, we compute [Measures].[Max Delta Artist %] as the maximum deviation across seasons for each artist and filter the result set to keep only the season where this deviation is maximal and strictly positive.

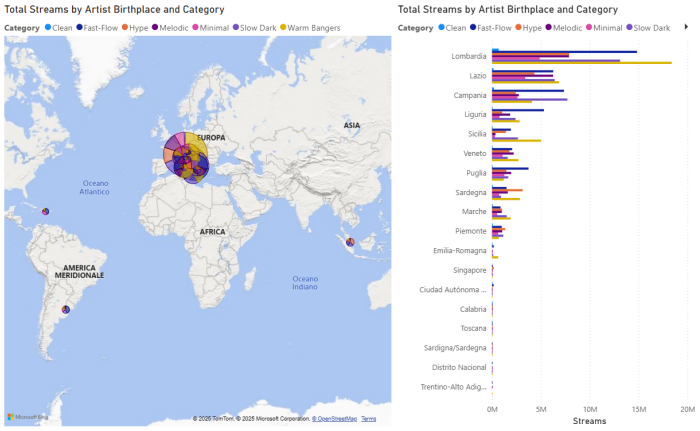

Figure 3.1: Geographic Distribution of Artists

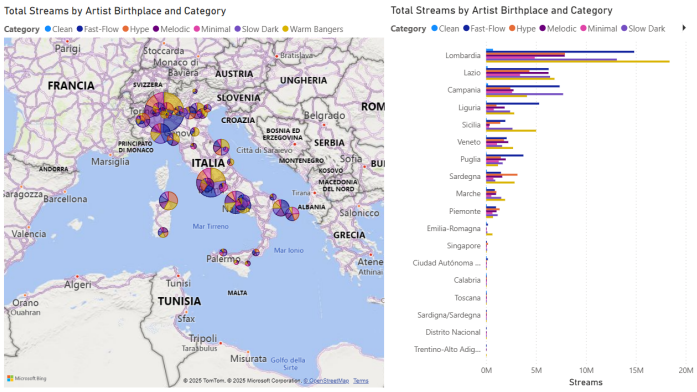

Figure 3.2: Geographic Distribution of Artists (Focus on Italy)

We believe this information is valuable because it clearly shows which season each artist tends to be listened the most over time. It can support release-timing decisions by helping identify when it makes the most sense to drop an album, aligning with audience listening habits and maximizing potential impact.

3.7 Assignment 20

This assignment was very straightforward and focused on the creation of a dashboard to visualize the total number of streams for each artist's birthplace, segmented by song category. We used two visualizations: a map, which allows for an immediate geographical understanding of where streams are concentrated, and a bar chart, which provides a clear comparison across categories and regions.

The combination of these visual elements makes it easier to localize patterns, apply filters by genre, and quickly identify differences in streaming performance across artists' birthplaces. The implementation of hierarchies in the cube now allows us to roll up (or drill down) and change the scope of our analysis.

3.8 Assignment 21

This assignment was more challenging compared to previous ones, mainly due to the limited availability of lyrics-related data in the cube and the fact that such information is confined to the *Dim Text* dimension. Despite these constraints, we were able to design a dashboard that provides meaningful insights into the relationship between lyrical content and song performance.

In particular, we created a scatter plot that displays individual songs in terms of total streams and the number of Italian swear words contained in their lyrics. This visualization allows us to directly observe how explicit language correlates with streaming performance at the song level. In addition, we implemented a stacked column chart to compare the distribution of swear words across different song genres, making it easier to identify genre-specific patterns.

Multiple insights emerge from these visualizations. Songs in the melodic genre show a non-negligible number of tracks with a high count of curse words, and these songs consistently perform poorly in terms of streams. This suggests that, within this genre, the use of explicit language may be detrimental and should likely be avoided.

More generally, limiting the number of swear words appears to be beneficial across all genres, although this effect is less pronounced for warm bangers.

Warm bangers, on the other hand, display a different behavior: they tend to achieve higher streaming performance when the number of swear words is very low, yet this genre also has the smallest proportion of completely non-explicit songs. This information can be particularly valuable for producers and artists operating in this genre, as it highlights a potential trade-off between stylistic conventions and commercial performance.

While exploring the data, we observed that the relationship between streams and swear word count appears to follow a power-law-like distribution. Although this observation is likely of limited business relevance, it's always nice to spot a power law.

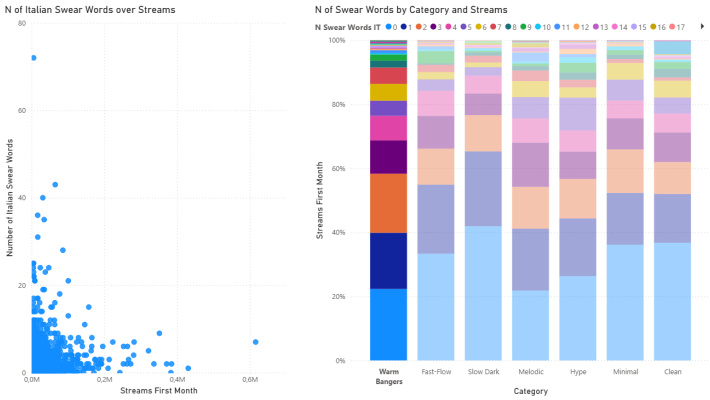

Figure 3.3: Swear Word Distribution with Focus on Warm Bangers

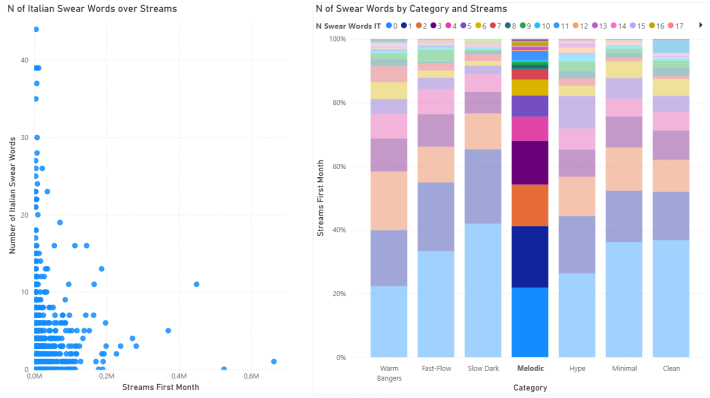

Figure 3.4: Swear Word Distribution with Focus on Melodic

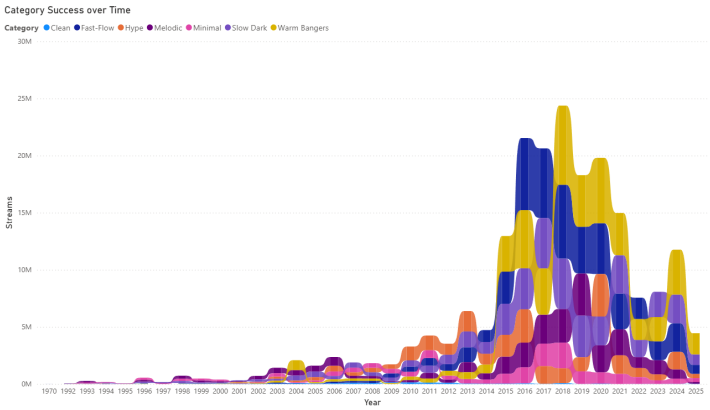

Figure 3.5: Total Streams by Category over Time

3.9 Assignment 22

This assignment was again quite straightforward and focused on illustrating how the popularity of different song categories evolves over time. To achieve this, we used a ribbon chart, which is particularly effective for highlighting changes in relative popularity and for showing how categories overtake each other across different time periods. This type of visualization allows for an immediate understanding of long-term trends, shifts in dominance, and periods of stability or change, making it well suited to communicate the temporal dynamics of category popularity.

An insight we can gather from the plot is that while warm bangers have consistently been on top of the chart in recent years, fast-flow seems to be losing ground to slow-dark.